Parallel Performance Analysis Using OpenMP

Aakriti Dhakal (a_d840)

***Abstract*—This report examines parallelizing the efficiency of calculating the value of Pi using OpenMP on the Lonestar6 high-performance system. The experiment evaluates impact of various optimization approaches, that includes thread placement methods, experiments with different numbers of parallel threads and variations in task scheduling. Using OpenMP's timing functions, the experiment measures performance metrics and delivers practical results for maximizing efficiency of maximizing parallel processing.**

I. Introduction

Serial computing executes instructions sequentially on a single processor, that leads to inefficient utilization of time and resources. These limitations are addressed by dividing tasks int smaller instructions and executing them across multiple processing units to improve performance. High-performance computing (HPC) relies on parallel computing to tackle large-scale tasks. One of the widely used shared memory parallel programming model facilitates parallel execution by distributing workloads across multiple processing units. This research experiments the effectiveness of different OpenMP parallelization techniques that focuses on:

- Thread placement strategies and their impact on performance
- Scheduling mechanisms and workload distribution
- Resource utilization patterns on multi-core architectures

The study aims to parallelize the "compute_pi.c" program using OpenMP and analyze runtime performance across various scheduling configurations. The experiments are conducted on the Lonestar6 cluster, which is a high-performance computing environment. The findings from this experiment contribute to the broader understanding of parallel computing optimization and provide practical guidelines for OpenMP implementation.

II. Implementation

A. Machine Architecture

For finding the architecture details of the machine, '*hwloc'* tool was used on a compute node of Lonestar6. This command was run on the idev node to gather the correct information on the machine architecture. The Lonestar6 node is equipped with two high-performance AMD EPYC 7763 64-Core processors, and results in a total of 128 cores across two sockets. These processors belong to CPU family 25, with a model number of 1 and stepping 1. The system is divided into two NUMA nodes: NUMA Node 0 includes cores 0 to 63, while NUMA Node 1 includes cores 64 to 127. The processor operates at a base frequency of 2.45 GHz and has the capacity to boost up to 3.53 GHz or scale down to 1.50 GHz according to the requirement. It has approximately 251 GB of available memory to handle large, memory-intensive workloads. It also supports AMD-V virtualization which enables efficient virtual machine deployment. Each core is equipped with a three-tiered cache system, consisting of a 32 KB L1 data cache for data and instruction respectively, a 512 KB L2 cache per core, and a large shared 32 MB L3 cache, that ensures efficient processing of memory-intensive tasks. These details provide information on how computation resources are structured that affects performance of OpenMP performance.

B. Parallelizing kernel using OpenMP

The "compute_pi.c" program was parallelized using OpenMP for finding the value of pi (π) efficiently. In a sequential form, the program iterates over a large number of intervals. By implementing OpenMP, the loop was parallelized to distribute workload among multiple threads.

The following OpenMP constructs were used in this program:

- *#pragma omp parallel for reduction(+:sum)* is used to parallelize the loop calculating Pi while ensuring correct summation
- *#pragma omp private(i)* declares loop index as private to each thread
- *#pragma omp_get_wtime()* measures execution time for analyzing performance improvements
- *omp_get_num_threads()* retrieves number of threads involved in parallel execution.

III. Performance Analysis

After implementing *compute_pi.c* program, the results were recorded in an *Excel* document and visualized using Matplotlib for better clarity. Graphs and line charts have been used to illustrate performance trends and how different factors impact performance.

A. Thread Placement Analysis

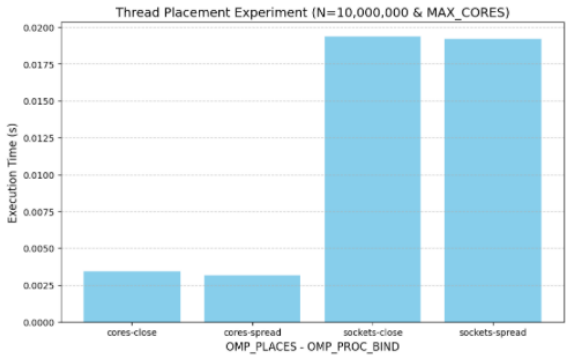

Fig 1: Thread Placement Experiment

As shown in Figure (1), thread placement has a significant impact on performance The experiment tested four configurations with N=10,000,000 iterations using the maximum number of cores (128): The observations showed the following result:

- Placing the thread at core level (OMP_PLACES=cores) outperforms socket-level placement, as they provide better data locality by keeping threads close to the data they access
- The 'spread' policy performs approximately 14% better than 'close', as it distributes threads more evenly across machine's topology
- Compared to core-level placement, socket-level placement executes almost 6.5 times slower as it leads to increased cross-socket communication

Based on these results, the optimal thread placement policy is cores-spread.

B. *Thread Scaling Analysis*

After conducting thread placement experiments, the optimal thread execution policy was identified as cores-spread. The impact of increasing the number of threads on execution time for N iterations was then analyzed. The thread count was scaled in powers of two, ranging from 1 to the maximum available cores, determined using the command *nproc --all.*

Figure (2) illustrates the variation in execution time as the number of threads increases. Key observations from the experiment include:

- The fastest execution time, approximately 0.002345 seconds, was achieved at 64 threads. Beyond this point, the execution time slightly increased with additional threads.
- The worst performance was noted at one thread with an execution time of approximately 0.037288 seconds. Using only 1 thread gave the highest execution time, showing that more threads significantly improve the performance of this application.

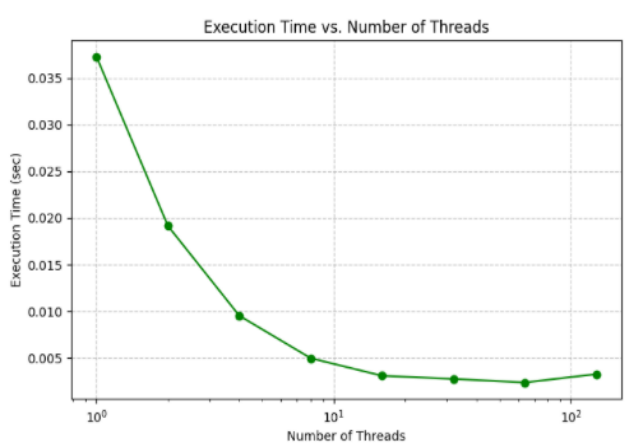

Fig 2: Number of Threads vs. Execution Time

C. *Scheduling Policy Analysis*

This experiment compared static and dynamic scheduling with varying chunk sizes (10, 100, 1000) using N=10,000,000 iterations, 128 threads, and cores-spread placement.

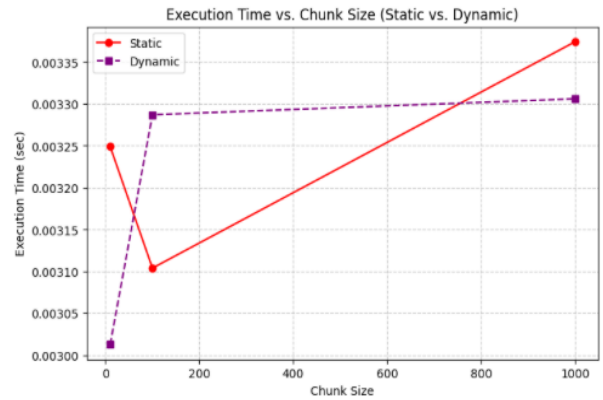

Fig 3: Scheduling Policies vs. Execution Time

Figure (3) compares the performance of static and dynamic scheduling using different chunk sizes. The key findings of this experiment are:

- Execution time remained relatively consistent across chunk sizes, with the best performance observed at a chunk size of 10. However, as the chunk size increased, execution time gradually rose, leading to slower performance.
- Performance varied significantly with different chunk sizes. The optimal execution time of 0.003287 seconds was achieved at a chunk size of 100, while both chunk size 10 and chunk size 1000 resulted in increased execution times.

IV. Conclusion

This experiment explored the optimization of parallel computing with OpenMP for the compute_pi.c program. Experiments with thread placement, thread count scaling, and scheduling strategies was done, and the results showed that core-level placement with 'spread' policy performed best, while thread scaling showed performance gains up to 64 threads, beyond which efficiency diminished due to overhead and resource contention.. Dynamic scheduling with a chunk size of 100 provided optimal load balancing. These findings highlight the importance of tuning OpenMP parameters to align with application workload and hardware architecture. Even for inherently parallel computations like π calculation, strategic configuration of thread placement, count, and scheduling can significantly improve performance.